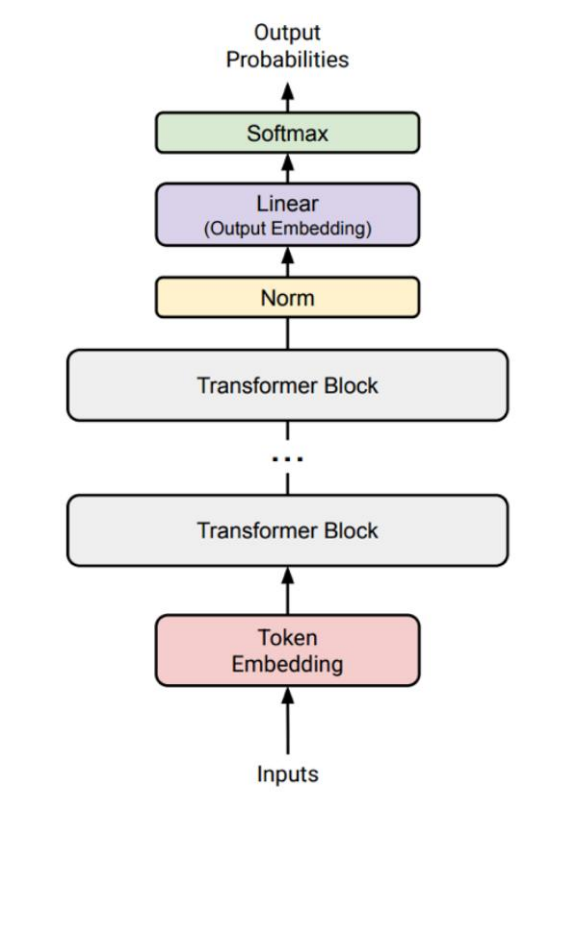
COMP7608B · LECTURE 1

Large Language Models

Foundations and inference

Wenbo Sun
School of Computing and Data Science
· HKU

DECODER-ONLY TRANSFORMER · ONE NEXT-TOKEN PASS



Architecture adapted from Hashimoto, CS336 Lecture 3

Your instructor

COURSE PERSPECTIVE

Research and industry shape this course.

- TU Delft PhD work — efficient LLM inference
- Database-system design for AI
- Fintech, Beijing — algorithm engineer
- Two years managing algorithmic trading

HOW THIS SHAPES THE COURSE

1. Understand the computation precisely enough to implement and measure it.
2. Treat memory, hardware and deployment constraints as design inputs.
3. Explain architecture choices clearly to engineers, managers and users.

The course connects mathematical mechanisms with the constraints of real systems.

WELCOME

About You



Two connected parts

PART I · 3 or 4 LECTURES

Inside the LLM

- Architecture, inference, training, and fine-tuning
- Small implementations; industry efficiency choices

- architecture and inference
- training and fine-tuning
- efficiency and deployment cost

PART II · 6 or 7 LECTURES

LLM as a tool

- Retrieval, tools, memory, and agent workflows
- Real systems and invited industry practitioners

- RAG and memory
- agent workflows and harnesses
- industry cases and failure modes

Why learn the internals?

WHY LEARN THE INTERNALS?

Understanding the computation transfers to new designs.

- Interviews may ask for formulas, code, or cost estimates
- Implementation reveals what the architecture really does
- New model design → identify which operations change
- Agent projects expose integration and evaluation problems

QUESTIONS YOU SHOULD BE ABLE TO REASON ABOUT

Why divide attention scores by $\sqrt{d_h}$?

Why does an FFN expand before contracting?

Which tensor or matrix dominates cost?

How would a new architecture change the code?

For agentic systems, implementation experience also exposes workflow traps, integration limits and evaluation problems.

Assessment: Group Project

WHAT COUNTS AS A PROJECT?

Build a working agentic application and defend its design.

- Start with a user problem and clear motivation
- Propose a stack; identify risks and alternatives
- Evaluate quality, reliability, latency, or cost
- Submit runnable code and explain limitations

POSSIBLE DIRECTIONS

- financial-news research assistant
- domain RAG with source verification
- data-analysis and visualization agent
- coding, testing or support workflow
- personal or organizational knowledge assistant

Benchmarking should support a design decision—for example, retrieval quality, latency, cost or tool-use reliability.

Scoring

20%

PROPOSAL

problem, motivation, provisional stack and risks

30%

PRESENTATION

show the system and defend its design

50%

REPORT + CODE

architecture, evaluation, limitations and runnable artifact

GROUPING Groups of 3–4. Form your team by 21 September; anyone without a group will be assigned randomly.

Detailed presentation arrangements and final deadlines will be announced after the teaching team completes the instructions.

AI assistance and ownership

PERMITTED ASSISTANCE

- AI tools and open-source libraries allowed
- Try guided exercises yourself first
- Explain your code and architecture choices

WHAT IS NOT ACCEPTABLE

1. Submitting another group's work or a complete copied project as your own.
2. Presenting code whose architecture and behavior you cannot explain.
3. Hiding external sources, datasets, generated content or major dependencies.

The presentation may include design questions or a live modification. If ownership is not demonstrated, marks may be deducted.

Let’s play a game

Starting from the left of a row, say one word, trying to keep the whole sentence meaningful.

Large	language	models	can	predict	…
In	Hong	Kong	the	weather	…

What context helped you choose your next word?

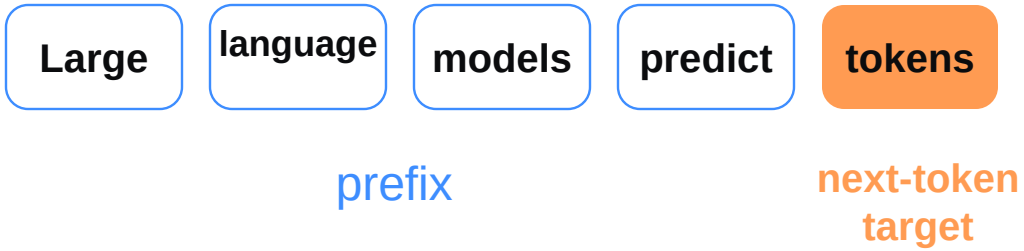
Language Models

FORMAL DEFINITION

A sequence probability is built from next-token probabilities.

$$P(x_1, \ldots, x_T) = \prod_t P(x_t \mid x_1, \ldots, x_{t-1})$$

- Each prediction depends on the existing prefix
- Chain rule combines the conditional probabilities
- Inference evaluates the next-token distribution
- Selection turns that distribution into a continuation



The model supplies probabilities; decoding selects the next token.

Language Models - Training

TRAINING VIEW

Predict the next token at every valid position.

- Inputs and targets are shifted by one token
- The target is the observed next token
- Future positions are hidden by a causal mask
- Known training positions can be processed together

Large	→ language
Large language	→ models
Large language models	→ predict
Large language models predict	→ tokens

$$\text{loss} = -\sum_t \log P(x_t \mid x_{<t})$$

The Starting Point

1951 · CLAUDE SHANNON

Likely events are less surprising.

- Surprisal measures information in one outcome
- Entropy averages surprise over a distribution
- Shannon studied prediction of printed English
- Next-token training uses a related cross-entropy objective

$$\text{surprisal}(x) = -\log_2 P(x)$$

$$\text{entropy} = \mathbb{E}[\text{surprisal}]$$

P(x)=1/2 → 1 bit

P(x)=1/8 → 3 bits

Cross-entropy and negative log-likelihood turn this idea into a trainable objective.

Shannon, “Prediction and Entropy of Printed English” (1951)

N-grams

STATISTICAL LANGUAGE MODELS

An n-gram sees only the previous n−1 tokens.

- Trigram example — predict from two preceding tokens
- Count frequent local combinations
- Rare combinations require smoothing or backoff
- Long dependencies remain outside the window

$$P(x_t \mid x_1, \ldots, x_{t-1}) \approx P(x_t \mid x_{t-2}, x_{t-1})$$



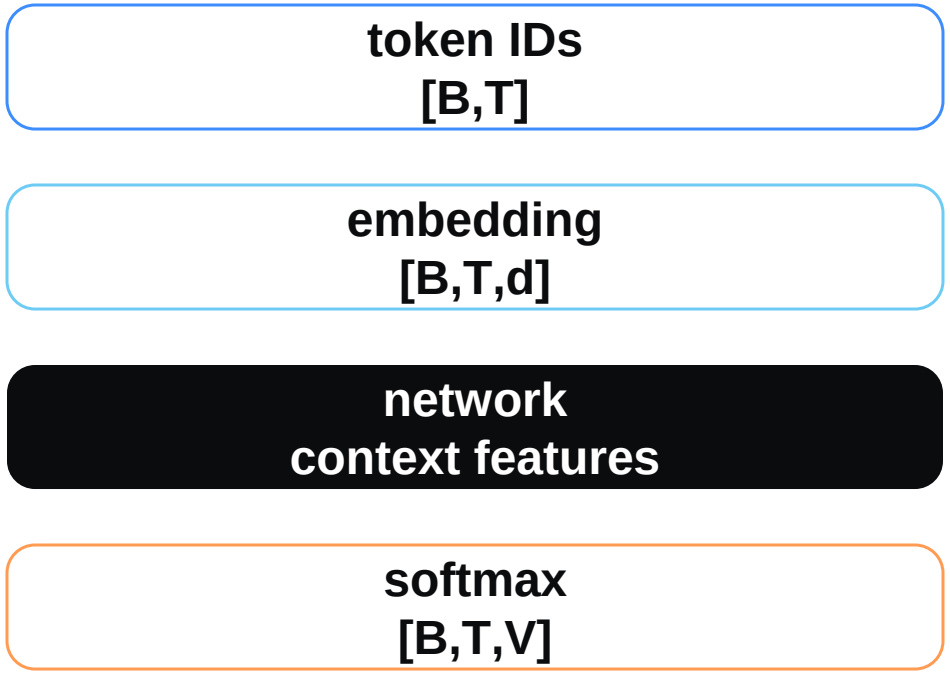
N-grams store local statistics explicitly; neural models learn continuous representations that can generalize across similar contexts.

Neural Language Models

2003 ONWARD

Learn token vectors and a shared predictor.

- Bengio et al. — neural prediction with learned word vectors
- Similar contexts can share statistical strength
- Word2vec later popularised embedding geometry
- Modern token embeddings are learned with the model



Modern LLMs learn token embeddings end to end, but embedding lookup is still the first learned operation in inference.

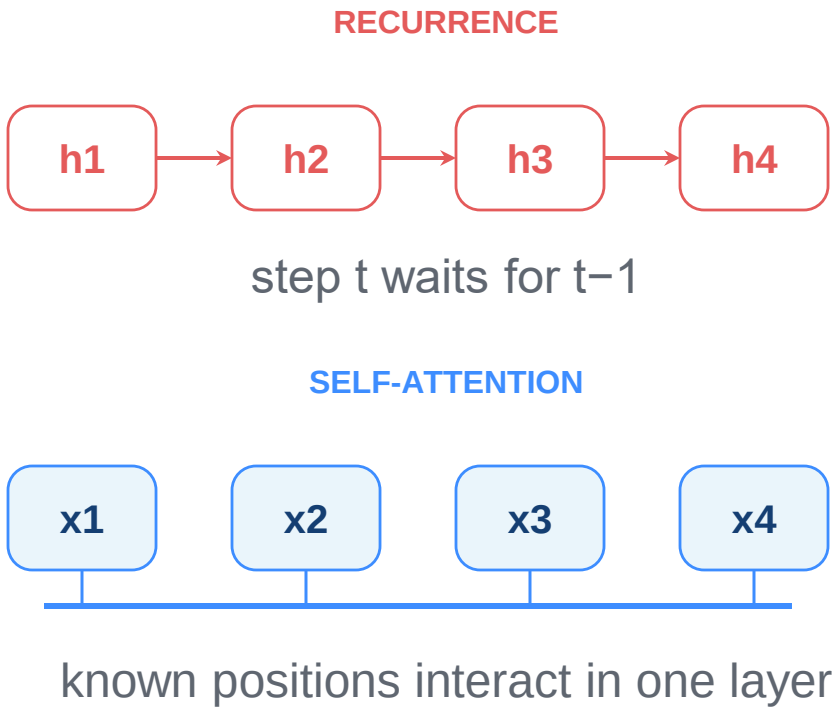
Bengio et al. (2003); Mikolov et al. (2013)

Transformers VS. RNN

2017 · TRANSFORMER

Self-attention removes the recurrent step dependency.

- RNN state at step t depends on earlier steps
- Transformer training already knows the input sequence
- Attention compares positions in parallel within a layer
- Dense attention introduces quadratic pairwise work



The Transformer did not make sequence modeling free; it exchanged recurrence for a highly parallel but potentially quadratic interaction.

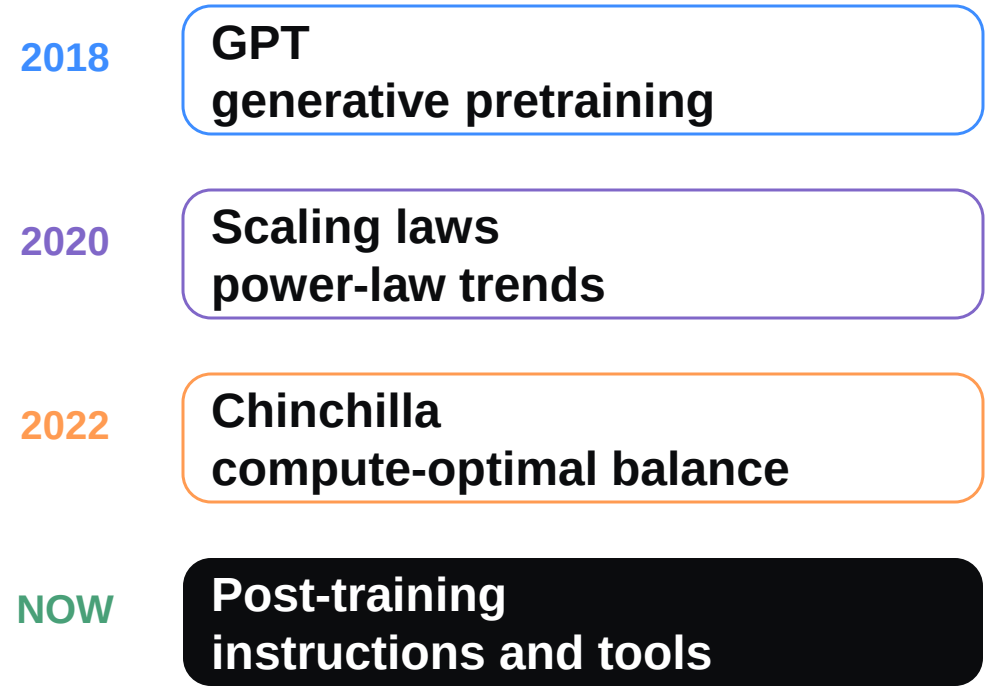
Vaswani et al. (2017), “Attention Is All You Need”

Generative pretraining and scaling laws

2018–2022

One training objective can support many downstream tasks.

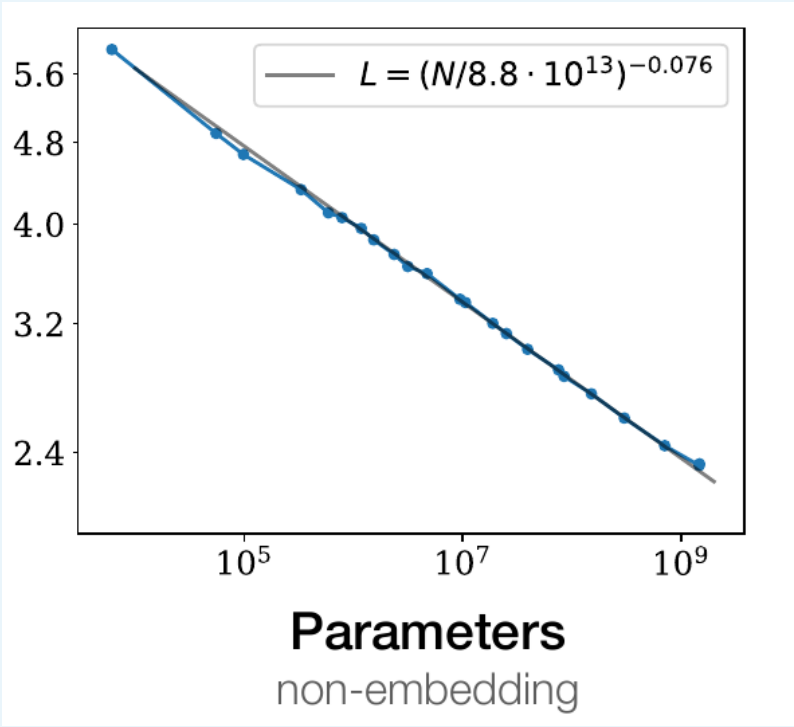
- GPT — reuse a generatively pretrained model
- Scaling studies relate loss to data, parameters, and compute
- Chinchilla — balance model size with training data
- Results depend on the data and training recipe



Scaling helps explain why larger training runs can improve loss, but it does not say that the largest model is always best for a particular domain or budget.

Radford et al. (2018); Kaplan et al. (2020); Hoffmann et al. (2022)

Scaling laws



Test loss ↓ · log–log axes

KAPLAN ET AL. · 2020

Prediction improves smoothly as training scales up.

- N — model parameters
- D — training tokens
- C — training compute (FLOPs)
- Power law → similar % loss reduction for each × increase in scale

Use small runs to forecast larger runs.
Keep model size, data and compute in balance.

Kaplan et al. (2020), Figure 1, model-size panel. Lower loss = better next-token prediction.

Balance model size with training data

FIXED TRAINING BUDGET

$C \approx 6 \times N \times D$

Dense-model training estimate
2× parameters at fixed compute
→ roughly ½ as many tokens

- More budget → grow N and D
- 2× N and 2× D → ≈4× compute
- ≈20 tokens/parameter: a guide*

HOFFMANN ET AL. · CHINCHILLA (2022)

4× smaller model

Gopher: 280B · 300B tokens
Chinchilla: 70B · 1.4T tokens

- Same training compute budget
- MMLU 5-shot: 60.0% → 67.6%
- Less compute per inference

***A training-compute guide, not a universal ratio.**
High serving demand can favour smaller models trained longer.

The landscape is now a systems story

THREE QUESTIONS FOR AN ENGINEER

What can we run, what does it cost, and what can we build?

- Model access and size
- Training and inference budgets
- Tools, data and agent control

OPEN WEIGHTS

Qwen 3.5 · Llama 4 · DeepSeek V4

MULTIMODAL INPUT

Text, images and documents together

REASONING EFFORT

More inference work before answering

AGENT HARNESSES

Context, tools, state and execution

A strong model is the starting point; useful applications need an engineered system.

Open weights span very different scales

SELECTED DOWNLOADABLE MODELS · B = BILLION · T = TRILLION

MODEL / FAMILY	TOTAL	ACTIVE	WEIGHT LICENSE
Qwen3.5-9B	9B	9B (dense)	Apache 2.0
Qwen3.5-35B-A3B	35B	3B	Apache 2.0
Llama 4 Scout	109B	17B	Llama 4 terms
Llama 4 Maverick	400B	17B	Llama 4 terms
DeepSeek V4 Flash	284B	13B	MIT
DeepSeek V4 Pro	1.6T	49B	MIT

Active counts per token; rounded LM sizes. V4 updates add a decoding module. Qwen figures exclude vision.

Open weights let you run a model; they do not necessarily reveal its training data.

Model size becomes a memory bill

18 GB

9B PARAMETERS

9 billion × 2 bytes
A dense-model scale

70 GB

35B PARAMETERS

35 billion × 2 bytes
Even if only 3B are active

3.2 TB

1.6T PARAMETERS

1.6 trillion × 2 bytes
A distributed-system scale

ASSUMPTION Illustrative 16-bit weight payload only. Actual checkpoints may use fewer bits.

MoE reduces active computation; the complete set of weights still needs storage.

Creating a model has several stages

Pretrain

LEARN FROM A CORPUS

Data → token batches
Next-token prediction
General capability

Post-train

SHAPE BEHAVIOUR

Demonstrations → SFT
Preference tuning + RL
Useful task behaviour

Serve

RUN REAL REQUESTS

Load and execute weights
Manage context and tools
Measure quality and cost

DIFFERENT BILLS

Data preparation, training runs, evaluation and serving consume different resources.

Training cost needs a clear boundary

DEEPSEEK-V3 · 2024 DISCLOSURE

$5.576M

Reported run-cost estimate
2.788M H800 GPU-hours
Assumed rental rate $2/hour

- Pretraining — 14.8T tokens
- 2,048 GPUs; under two months
- Prior research / ablations excluded

LLAMA 3.1 405B · 2024 DISCLOSURE

$61.7–123.4M*

Our illustrative rental estimate
30.84M H100 GPU-hours
Assume $2–4/GPU-hour

- *Not a bill reported by Meta
- ≈78 days at 16,384 GPUs†
- †Idealised GPU-hour conversion

GPU-hours measure total work. Calendar time and the full development budget differ.

API prices are quoted per million tokens

USD PER 1M TOKENS · TEXT · UNCACHED INPUT · CHECKED 8 SEP 2026

MODEL / SERVICE	INPUT	OUTPUT	PRICE BASIS
DeepSeek V4 Flash	$0.22	$0.66	Off-peak
DeepSeek V4 Flash	$0.44	$1.32	Peak
Gemini 3.8 Flash	$0.75	$3.75	Introductory*
Claude Sonnet 5	$2.00	$10.00	Standard
Claude Opus 5	$5.00	$25.00	Standard

*Gemini rate through 31 Dec 2026. Batch, cache, tools and taxes are excluded. Prices can change.

At $10 per million output tokens, one output token costs $0.00001.

Budget the complete task

ONE RESPONSE · SONNET 5 EXAMPLE

$0.009 / task

2,000 input + 500 output tokens
$0.004 input + $0.005 output
10,000 tasks → $90

- Input $2 / 1M; output $10 / 1M
- All input uncached in this example
- No paid tools or other fees included

EIGHT-CALL AGENT · SAME RATES

$0.10 / task

30,000 input + 4,000 output
$0.060 input + $0.040 output
10,000 tasks → $1,000

- Input grows 2,000 → 5,500 / call
- 500 output tokens on each call
- ≈11× the single-response cost

Task cost = all input charges + all output charges + tools and infrastructure.

Three common application architectures

RAG

RETRIEVE → ANSWER

Search a document index
Pass evidence to the LLM
Answer with references

Tool agent

ACT → OBSERVE

LLM selects a tool call
Code executes the tool
LLM uses the result

Multi-agent

DELEGATE → COMBINE

Delegate subtasks
Run workers in parallel
Combine their results

CONTROL A fixed workflow chooses steps in code. An agent chooses some steps dynamically.

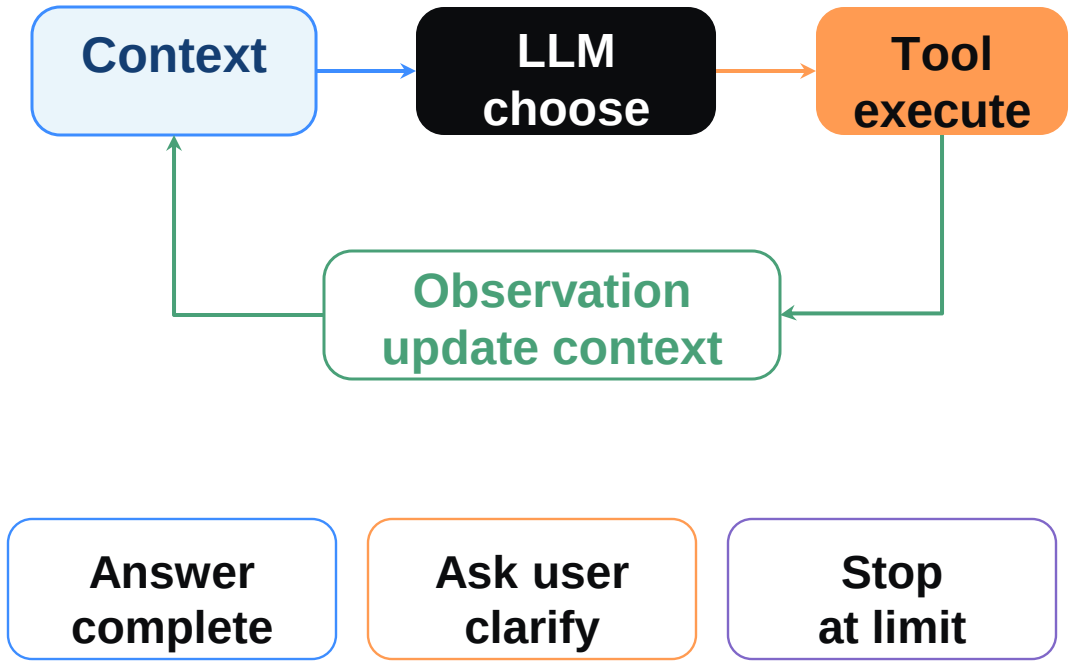
RAG can be a tool inside an agent. Multiple agents add coordination and token cost.

An agent is a controlled execution loop

THE HARNESS RUNS AROUND THE MODEL

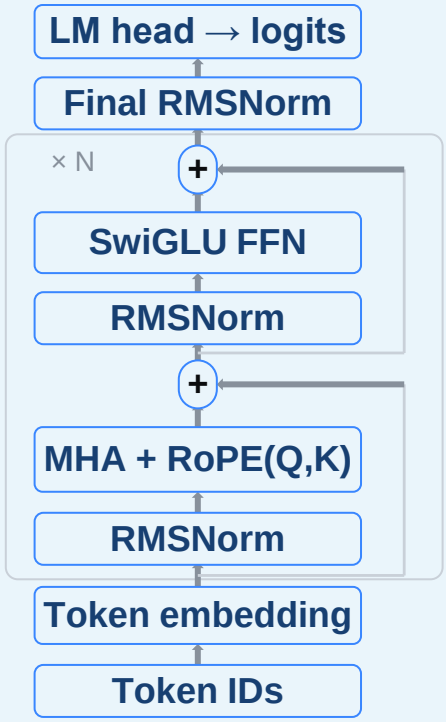
The model proposes an action; software executes it.

- Context — instructions, evidence, state
- Tools — search, code, business APIs
- Runtime — permissions, retries, limits
- Results — observations for the next call



A tool call is structured model output. The application supplies the actual action.

Today — one prompt, one next token



We will return to this map throughout the lecture.

ONE ORGANIZING QUESTION

What happens between a prompt and its next token?

1. text → token IDs
2. vectors → decoder blocks
3. hidden state → logits → sampled token

Tokens are the model’s text units

- A token is one minimum unit of a language.
- A token ID is the integer label for that unit.

Text	One possible split	Token IDs
`lower`	`low` · `e` · `r`	`257, 101, 114`

- Tokens may be words, word fragments, spaces or individual bytes.

The tokenizer chooses the units. The model receives their IDs.

Why choosing tokens is a trade-off

English

`unhappiness, GPT-6, 3.14`

New words, names and numbers keep appearing.

Chinese

我喜欢机器学习

Word boundaries are usually not marked by spaces.

Bytes

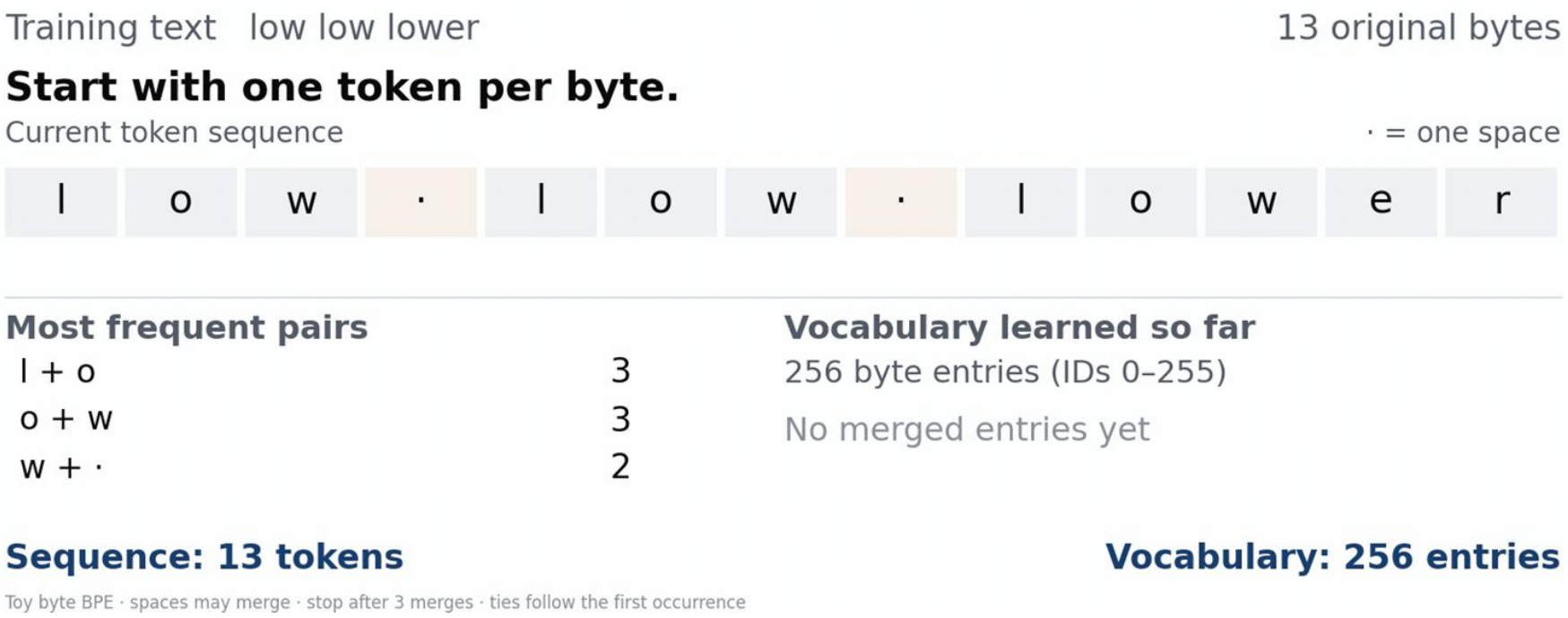
`cat → 63 61 74`　　猫 → `E7 8C AB` (hex)

256 byte values cover UTF-8; byte sequences can be long.

> **Compression gives us compact units while retaining byte coverage.**

Byte-pair encoding (BPE)

Count adjacent pairs → merge the most frequent → repeat



Gage (1994): compression origin · Sennrich et al. (2016): subword tokenization

Train once, reuse the learned merges

Learn once

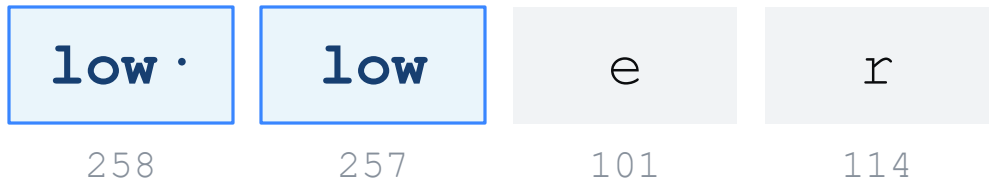
Rank	Merge	New ID
1	`l + o → lo`	256
2	`lo + w → low`	257
3	`low + · → low·`	258

Space merges depend on pre-tokenization rules.

Apply many times

- Use the saved merges in priority order.
- Create no new vocabulary entries.

`New text: low lower`



Use the trained tokenizer; its token IDs select embedding rows.

Vocabulary size trades sequence length for storage

TOKENIZER HYPERPARAMETER

Shorter sequences require a larger embedding vocabulary.

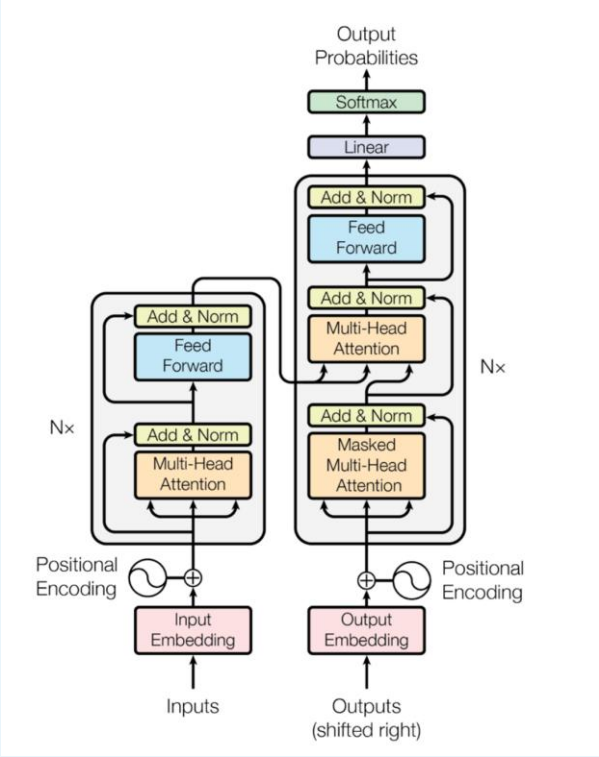
- Larger V can reduce the number of tokens T
- Input embedding stores V × d weights
- An untied output head stores another d × V
- Rare vocabulary pieces receive less training

SMALLER V	LARGER V
1 longer token sequences	1 shorter token sequences
2 smaller V×d matrices	2 larger V×d matrices
3 more byte fragmentation	3 more rare pieces

```
embedding params = Vd    ·
untied input + output ≈ 2Vd
```

Multilingual tokenizers often use larger vocabularies because useful pieces must be shared across many scripts and languages.

Transformer



2017 ARCHITECTURE

The encoder reads the source; the decoder predicts the target.

- Encoder output — one contextual vector per source token
- Decoder self-attention reads the target prefix
- Cross-attention reads encoder outputs as K and V
- Examples — T5, BART, and translation models

Examples include T5, BART, MarianMT and many translation models.

Vaswani et al. (2017), Figure 1. Original figure reproduced in the supplied CS336 lecture.

Our toy decoder configuration

TEACHING INSTANCE

One small configuration for shapes and cost calculations.

- One block with four independent MHA heads
- RMSNorm, RoPE, and a SwiGLU FFN
- $d = H \times d_h = 4 \times 32 = 128$
- d_ff = 344; input and output embeddings tied

HYPERPARAMETER	SYMBOL	VALUE
batch	B	1
sequence	T	16
vocabulary	V	2,048
model width	d	128
blocks	N	1
heads	H	4
head width	d_h	32
SwiGLU width	d_ff	344

Teaching assumptions: bias-free projections, MHA, pre-RMSNorm, RoPE, SwiGLU, tied embeddings.

One rule for counting matrix-multiply FLOPs

THE REUSABLE RULE

Count the outputs, then the work per output.

- [m,k] × [k,n] produces m × n outputs
- Each output is a length-k dot product
- One multiply + one add count as two FLOPs
- Leading cost ≈ 2mkn

$$[m,k] \times [k,n] \rightarrow [m,n]$$

Example — three QKV projections

One projection
$[B,T,d] \times [d,d] \rightarrow [B,T,d]$
$2BTd^2$ FLOPs

Three projections = $6BTd^2$
B=1, T=16, d=128 → 1.573 MFLOPs

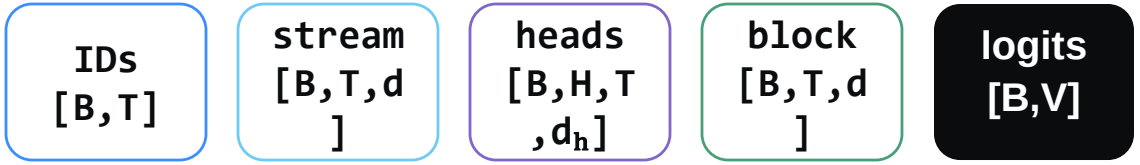
Leading matrix counts omit biases and smaller elementwise operations; weight storage is counted separately.

Tensor shapes through the decoder

SHAPE SCAFFOLD

Track temporary shapes as well as the residual stream.

- Residual stream stays [B,T,d] between blocks
- Attention introduces a head axis and pairwise scores
- FFN temporarily expands d to d_ff
- Generation selects the final position before the LM head

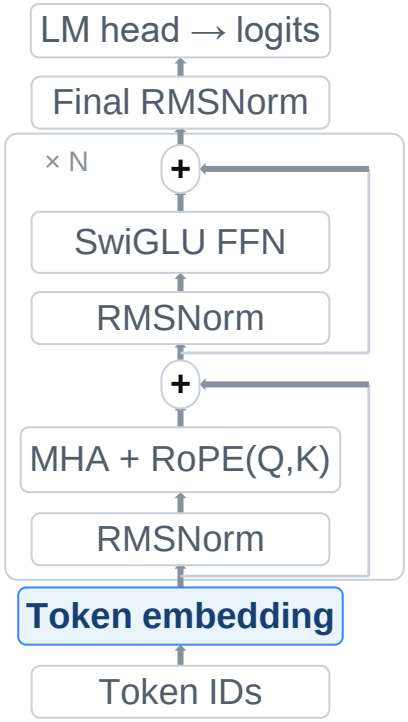


temporary shapes determine memory and FLOPs

[1,16] → [1,16,128] → … → h_T [1,128] → [1,2048]

Whenever two tensors are added by a residual connection, their shapes must match.

Embedding lookup selects learned rows



FROM INTEGERS TO VECTORS

Token ID i selects row i of the embedding table.

- E has V rows and d learned features per row
- Gather rows in the order of the input IDs
- Repeated IDs initially select identical vectors
- Training learns the table; inference only reads it

Check — same ID twice: same initial vector?

$$E \in \mathbb{R}^{V \times d} \qquad IDs\ [B,T] \rightarrow X = E[IDs]\ [B,T,d]$$

Separate small example — E [3,4]

ID	f_1	f_2	f_3	f_4
0	.2	.1	0	−.1
1	1	0	.5	−.5
2	0	1	−.5	.5

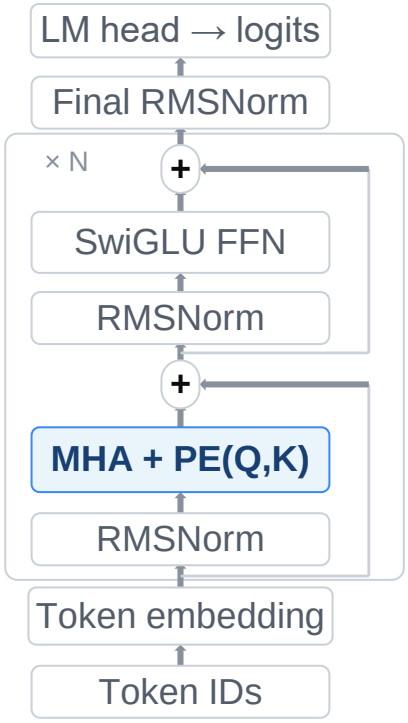
IDs [1,2,1] → select rows 1, 2, 1

1	0	.5	−.5
0	1	−.5	.5
1	0	.5	−.5

INPUT	OUTPUT	PARAMETERS	FLOPs
IDs [1,16]	X [1,16,128]	Vd = 262,144	≈0 · gather

Embedding lookup

Positional Encoding



MISSING COORDINATE SYSTEM

Order and distance help distinguish contextual roles.

- Token identity alone does not specify relative distance
- A dog bites a man; a man bites a dog
- Original model — add positional vectors to embeddings
- Our model — inject position inside attention with RoPE

a **dog** bites **man**

a **man** bites **dog**

same words · different order · different meaning

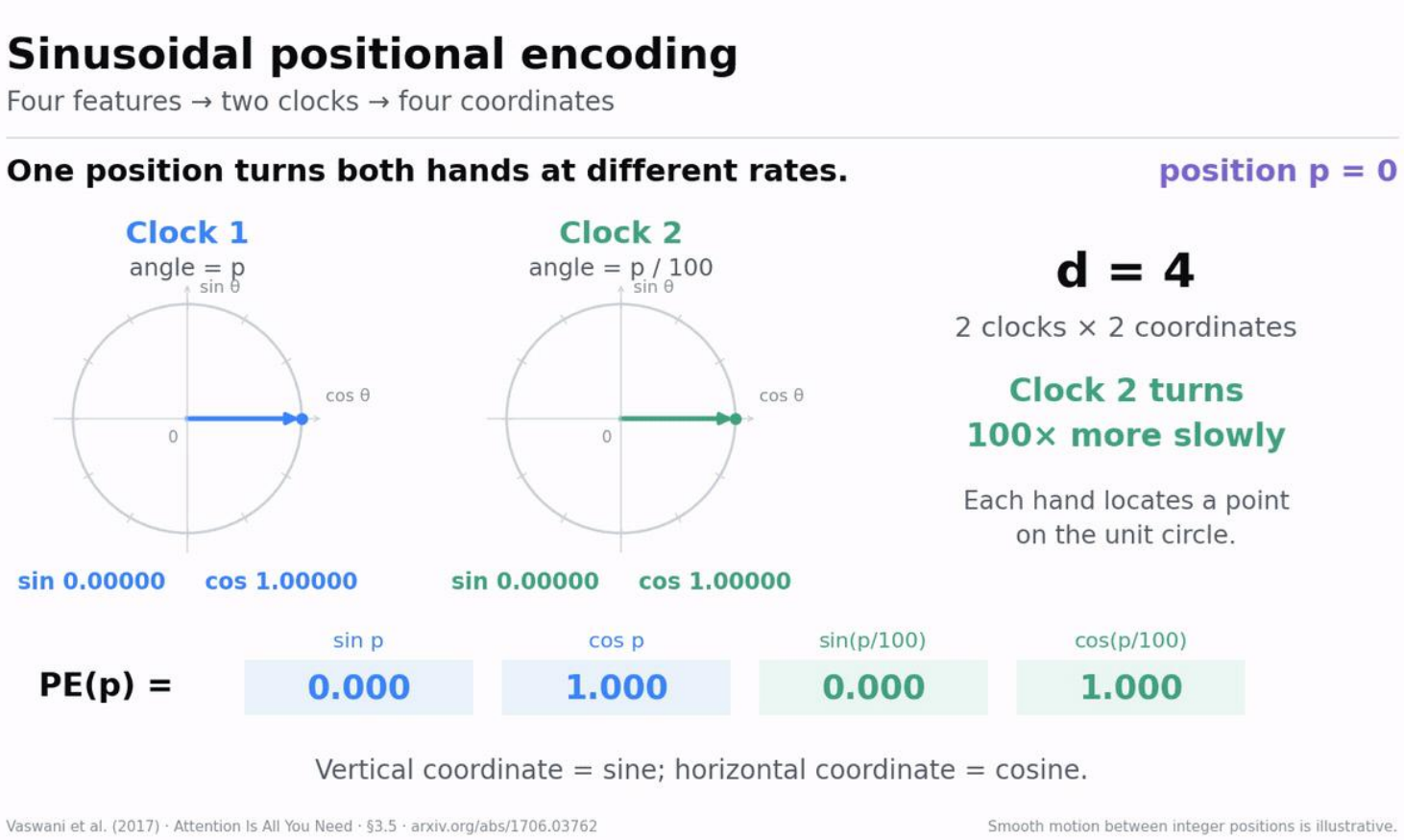
Original Transformers add position to embeddings; many recent decoder models rotate Q and K with RoPE.

Sinusoidal Positions Encoding

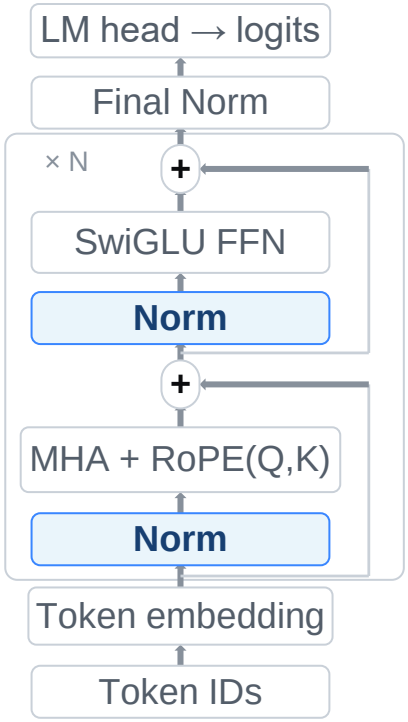
ORIGINAL TRANSFORMER

Two coordinates locate one hand; different clocks rotate at different rates.

- d = 4 ⟶ two clocks; positions p = 0, 1, 2
- Clock 1 angle p; clock 2 angle p/100
- Compute sin and cos for every position
- Add PE [3,4] to embeddings X [3,4] once



LayerNorm



GENERAL NORMALIZATION IDEA

Compute statistics across the features of one token.

- Subtract that token's feature mean
- Divide by its feature standard deviation
- Apply learned scale γ and offset β
- No mixing across tokens; same [B,T,d] shape

$$LN(x)=\gamma \odot (x-\mu)/\sqrt{(\sigma^2+\varepsilon)} + \beta$$

Same vector for both normalization examples

$x = [1, 2, 3, 4]$

$\mu = 2.5 \qquad \sigma^2 = 1.25$

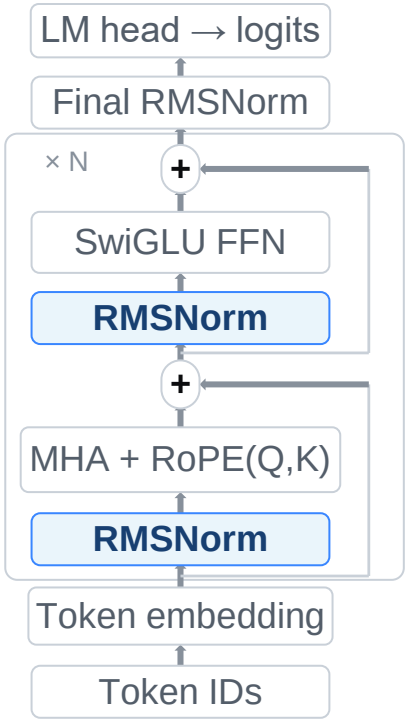
$LN(x) \approx [-1.34, -0.45, 0.45, 1.34]$

Here γ=1, β=0; tiny ε omitted in the arithmetic.

INPUT	OUTPUT	PARAMETERS	FLOPs
[1,16,128]	same shape	2d = 256	≈7BTd

Ba et al. (2016)

RMSNorm



COMMON MODERN REPLACEMENT

Divide by root-mean-square magnitude; apply learned γ.

- Compute the mean of squared features
- Divide by its square root, with ε for stability
- Keep learned γ; omit mean subtraction and β
- Same shape; one learned parameter per feature

$$\mathrm{RMSNorm}(x)=\gamma \odot x / \sqrt{(\mathrm{mean}(x^2)+\varepsilon)}$$

Compare using x = [1, 2, 3, 4]

$\mathrm{mean}(x^2) = 7.5$
$\mathrm{RMS}(x) = \sqrt{7.5} \approx 2.74$

$\mathrm{RMSNorm}(x) \approx [0.37, 0.73, 1.10, 1.46]$

Here γ=1; ε omitted. The mean is not forced to zero.

INPUT	OUTPUT	PARAMETERS	FLOPs
[1,16,128]	same shape	d = 128	≈4BTd

Zhang & Sennrich (2019)

Pre-norm or Post-norm

PLACEMENT CHOICE

Each branch adds an update to the existing hidden state.

- Residual addition — x + F(x), with matching shapes
- Post-norm — normalize after adding the update
- Pre-norm — normalize only the branch input
- Our decoder uses pre-norm and a final RMSNorm

POST-NORM · ORIGINAL

```
y = Norm(x + F(x))
```

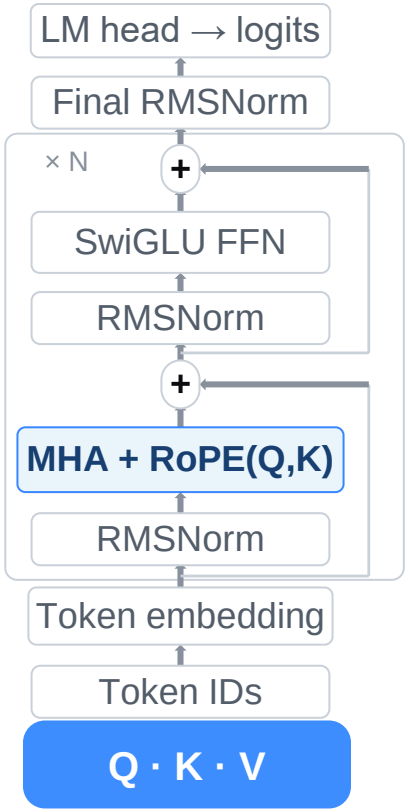
PRE-NORM · MODERN

```
y = x + Norm(F(x))
```

Pre-norm often improves optimization stability; it does not guarantee a better final model under every training recipe.

Xiong et al. (2020)

Queries, keys, and values



THREE LEARNED VIEWS

Three learned views of the same hidden state.

- Q — what information this position seeks
- K — how another position can match it
- V — the information it can contribute
- Every token produces all three; features are learned

Q — **Query**

What information do I need?

K — **Key**

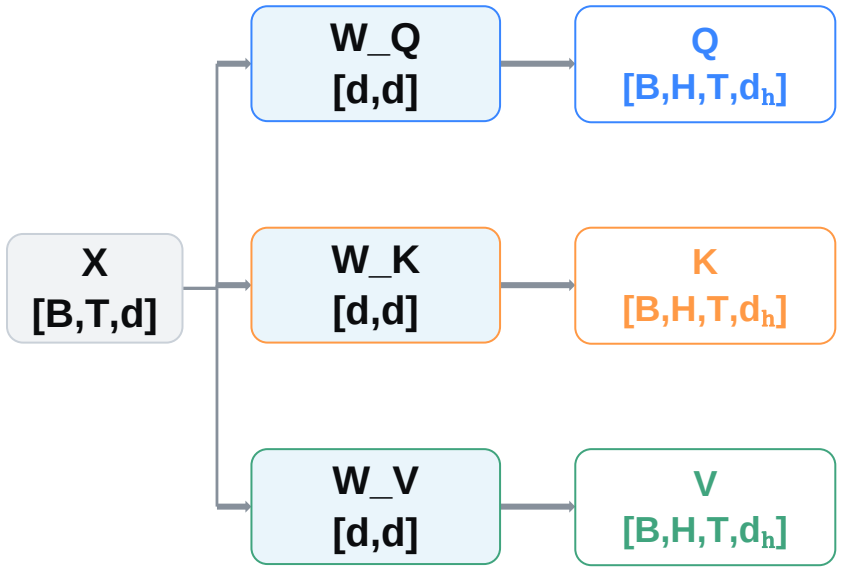
When should another query select me?

V — **Value**

What do I contribute if selected?

A high Q–K score decides where to read; V decides what information is read.

Project QKV, then split into heads



Each projection keeps width d before the head reshape.

One projection, then reorganize features

Project XW_Q

[B,T,d] = [1,16,128]

Reshape $d = H \times d_h$

[B,T,H,d_h] = [1,16,4,32]

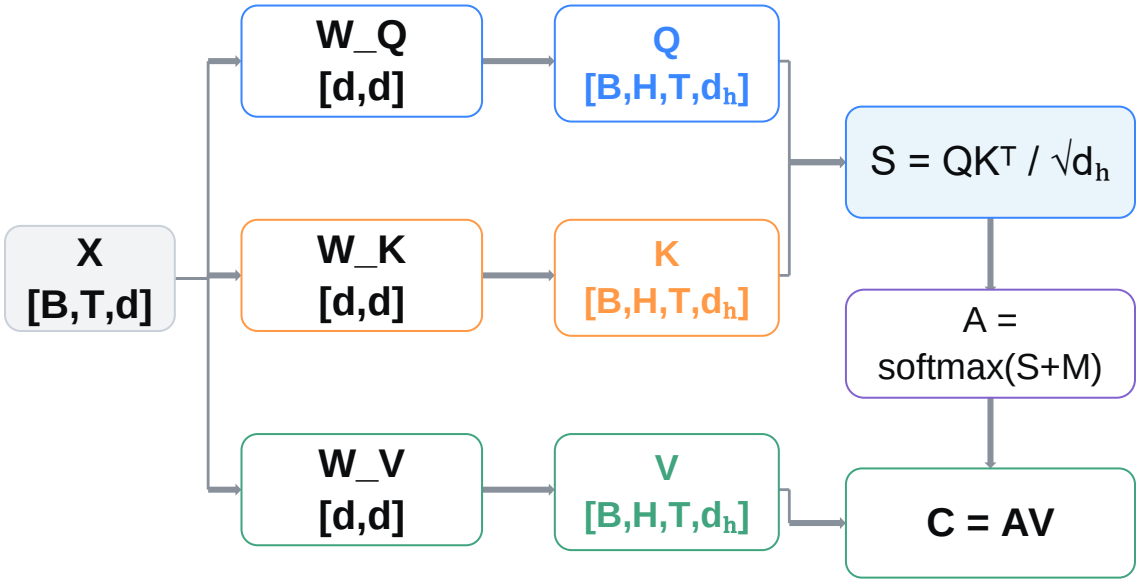
Transpose Move the head axis

[B,H,T,d_h] = [1,4,16,32]

Do the same for K and V. Next, rotate Q and K.

INPUT	OUTPUT	PARAMETERS	FLOPs
X [1,16,128]	3× head tensors	$3d^2$ = 49,152	$6BTd^2$

Queries and keys produce pairwise scores



Q and K include RoPE; M excludes future positions.

$$S = QK^T / \sqrt{d_h}$$

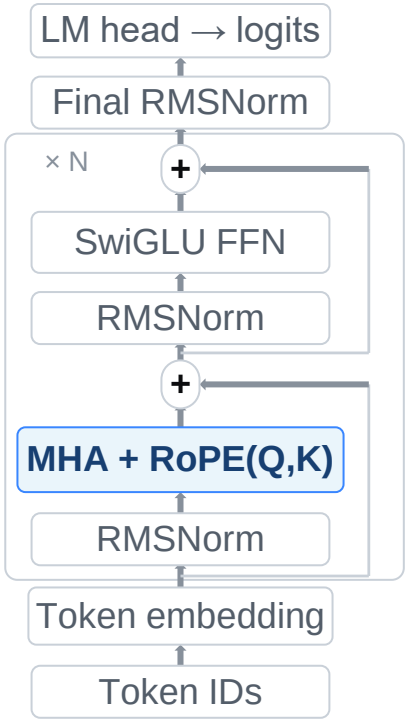
$$Q_{[1,4,16,32]} \times K^T_{[1,4,32,16]} = S_{[1,4,16,16]}$$

For T=16, each head contains 16×16 pair scores.

Each row compares one query with all keys. Add M before softmax to hide future positions.

INPUT	OUTPUT	PARAMETERS	FLOPs
Q,K head tensors	S [1,4,16,16]	0	$2BT^2d$

Rotary Positional Encoding (RoPE)



RoPE makes attention sensitive to relative position

Hold token content fixed. Move the tokens. Watch their pair score.

Where is the key relative to this query?

Token positions	0	1	2	3	4	5	6	7	8	9	10
			key K: cat			query Q: slept					

key is 3 positions before query

	Content vectors held fixed					After RoPE			
K	-0.600	-0.800	-0.200	0.400	RoPE at position 2	0.977	-0.213	-0.208	0.396
Q	0.800	0.600	0.300	-0.100	RoPE at position 5	0.802	-0.597	0.305	-0.085

Attention pair score

0.407

dot product / √4 · before softmax

Reference score

RoPE mixes each pair of Q/K features using position-dependent sine and cosine values.

Here, the key is three positions before the query.

Su et al. (2021) · RoFormer · §3.2 · arxiv.org/abs/2104.09864

Illustrative Q/K, not model predictions. Smooth movement illustrates integer positions.

Score Scaling

NUMERICAL STABILITY

Divide scores by $\sqrt{d_h}$ before softmax.

- A dot product sums d_h products
- Under simple assumptions, its variance grows with d_h
- Large scores can make softmax overly concentrated
- Scaling controls variance without changing shape

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}$$

$$\text{Var}(q \cdot k) \approx d_h \Rightarrow \text{Var}(q \cdot k / \sqrt{d_h}) \approx 1$$

SOFTMAX EXAMPLE

large scores

Scores [4, 1, 0]
Probabilities
93.6%, 4.7%, 1.7%

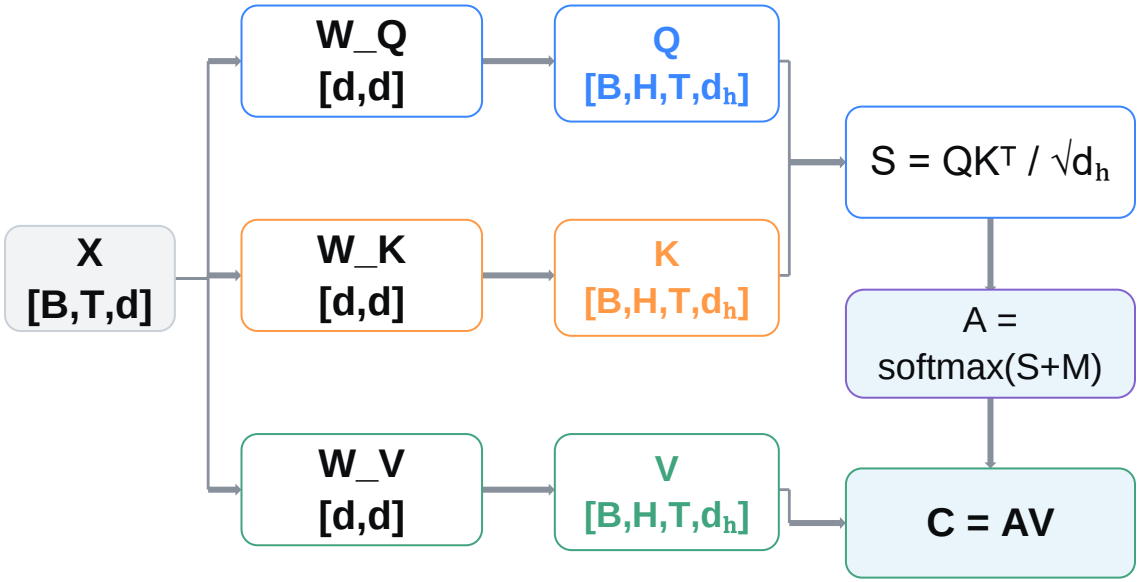
controlled scores

Scores [2, 0.5, 0]
Probabilities
73.6%, 16.4%, 10.0%

Example $d_h = 4$, so divide scores by 2. Softmax changes; tensor shape stays fixed.

INPUT	OUTPUT	PARAMETERS	FLOPs
S [B,H,T,T]	Scaled S; same shape	0	lower-order

Attention Scores



Q and K include RoPE; M excludes future positions.

Check — change V only: do the weights change?

One head; three visible keys; values have width 2

S row = [2, 1, 0]

A = softmax(S+M) ≈ [0.665, 0.245, 0.090]

key	weight	value
1	0.665	[2,0]
2	0.245	[0,2]
3	0.090	[1,1]

C = 0.665[2,0] + 0.245[0,2] + 0.090[1,1]
≈ [1.420, 0.580]

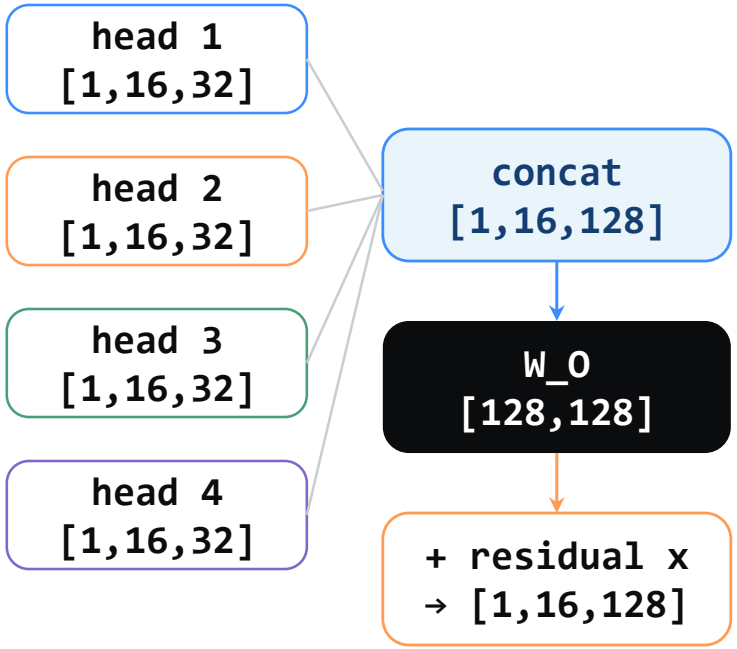
INPUT	OUTPUT	PARAMETERS	FLOPs
A [B,H,T,T], V	C [B,H,T,d_h]	0	$2BT^2d$

QKV Concat

RETURN TO MODEL WIDTH

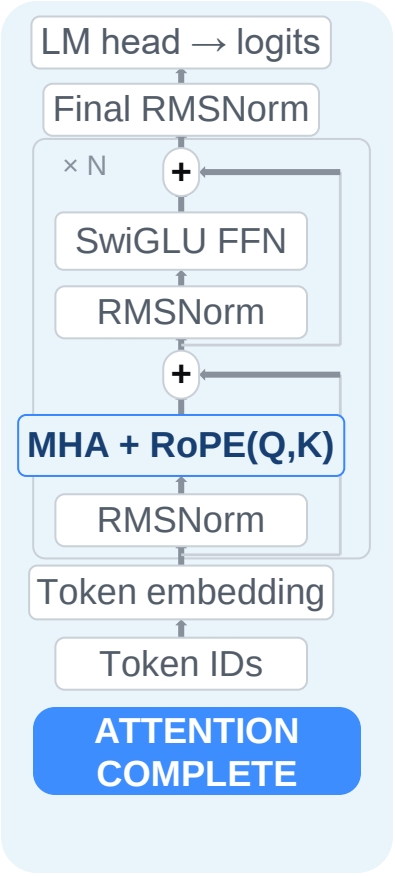
Concatenation restores width d; W_O mixes heads.

- Each head returns [B,T,d_h]
- Concatenate H heads into [B,T,d]
- Apply W_O [d,d] to mix their outputs
- Add the result to the unchanged residual input



INPUT	OUTPUT	PARAMETERS	FLOPs
4×[1,16,32]	[1,16,128]	W_O: d^2=16,384	$2BTd^2$

Attention Computing Cost



LAYER CHECKPOINT

Dense projections grow with T; pairwise work grows with T^2.

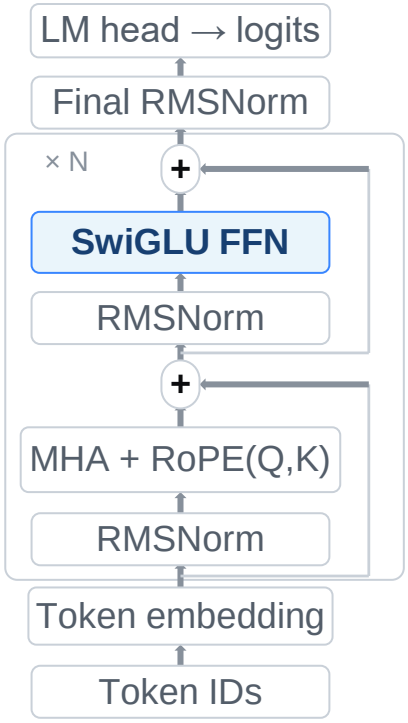
- QKV and output matrices contribute $8BTd^2$ FLOPs
- Score and value products contribute $4BT^2d$
- Short toy context — dense projections dominate
- Long context — pairwise work grows faster

$$\text{MHA FLOPs} \approx 8BTd^2 + 4BT^2d$$

QKV projections	$6BTd^2$	1.573M
$QK^T + AV$	$4BT^2d$	0.131M
output projection	$2BTd^2$	0.524M

$$\text{parameters} = 4d^2 = 65{,}536$$

Activation: Rationale



NEURAL-NETWORK FOUNDATION

Purely affine layers collapse into one affine map.

- Two affine maps combine into another affine map
- An activation adds nonlinear feature responses
- Depth can then express more complex transformations
- This argument concerns the FFN's affine layers

$$W_2(W_1x+b_1)+b_2 = (W_2W_1)x + (W_2b_1+b_2)$$

WITHOUT ACTIVATION

linear → linear → still linear

WITH ACTIVATION σ

linear → σ → nonlinear features

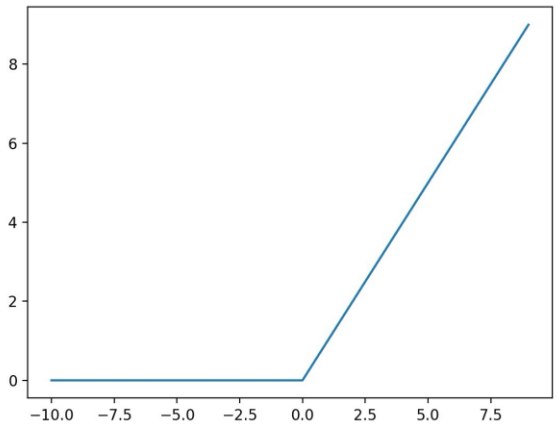
Linear layers mix features; activations allow the network to construct new nonlinear features.

Activation: ReLU

ORIGINAL TRANSFORMER

A separate feature transformation at every token position.

$$FFN(x)=ReLU(xW_1+b_1)W_2+b_2$$



ReLU(z) = max(0,z)

x	up W_1	ReLU	down W_2
[1,16,128]	[1,16,512]	same	[1,16,128]

4d is an empirical convention; cost depends on the chosen intermediate width.

INPUT	OUTPUT	PARAMETERS	FLOPs
[1,16,128]	[1,16,128]	$8d^2$=131,072	4BTd·d_ff

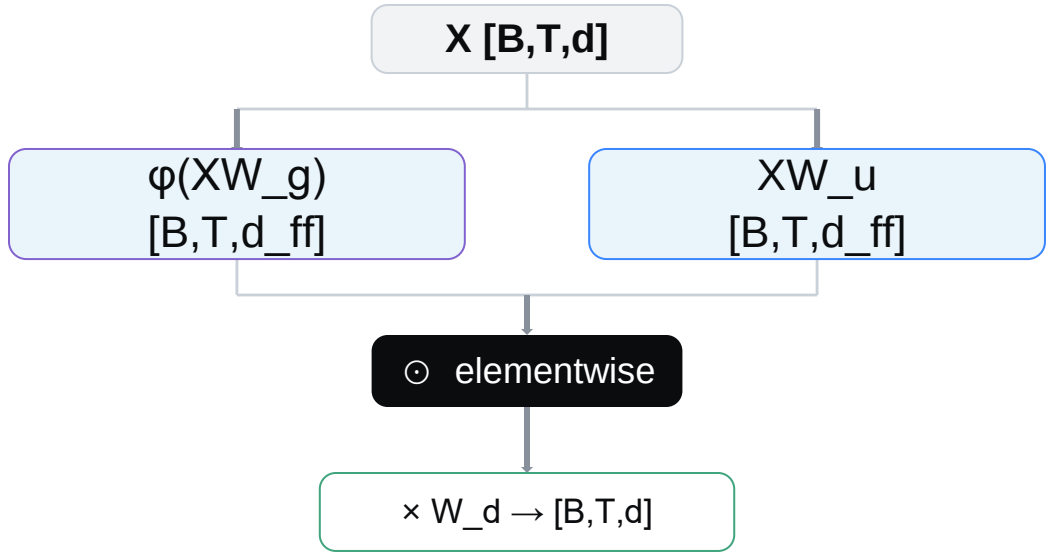
Gated ReLU (*GLU)

TWO INTERMEDIATE BRANCHES

Both branches receive the same input.

- Content branch proposes features
- Gate branch modulates those features
- Multiply the branches elementwise
- A third matrix projects the result back to d

$$FFN(X) = [\varphi(XW_g) \odot (XW_u)]W_d$$



ReGLU uses ReLU; GEGLU uses GELU; SwiGLU uses SiLU on the gate branch.

Shazeer (2020)

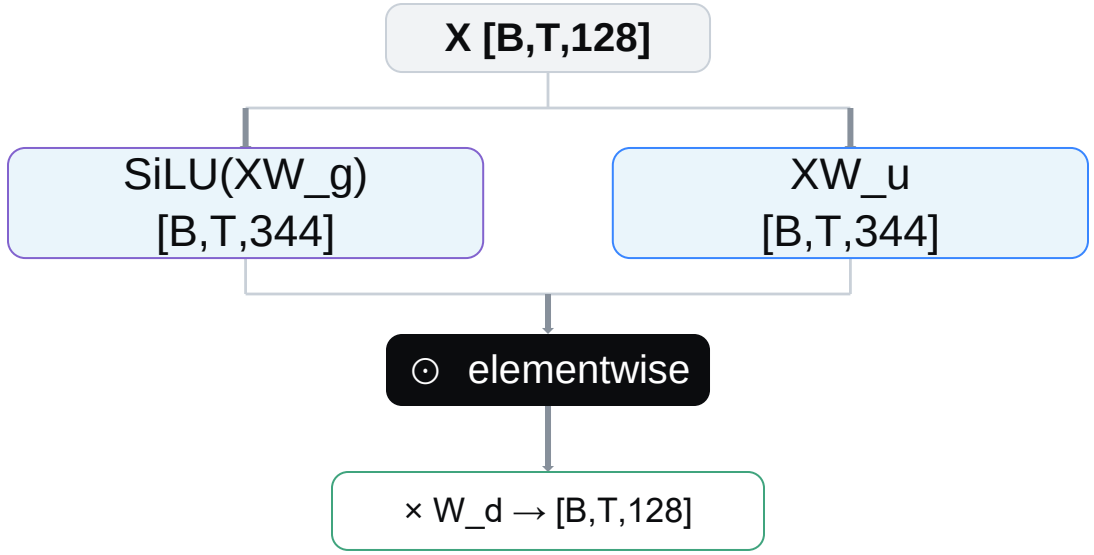
SwiGLU

COMMON MODERN CHOICE

SiLU(z) = z × sigmoid(z); also called Swish with β = 1.

- Gate and content projections each produce 344 features
- SiLU changes the gate values without changing shape
- Elementwise product stays 344 wide
- Down projection returns to d = 128

$$\mathrm{SwiGLU}(X) = [\mathrm{SiLU}(XW_g) \odot (XW_u)]W_d$$



INPUT	OUTPUT	PARAMETERS	FLOPs
[1,16,128]	[1,16,128]	3d·344=132,096	6BTd·344

PyTorch SiLU / Swish

FFN Expansion Ratios

CAPACITY ALLOCATION

The 8/3 ratio matches a three-matrix FFN to a two-matrix 4d FFN.

- Plain FFN — 2d × d_ff parameters
- Gated FFN — 3d × d_ff parameters
- Matching budgets gives d_ff = 8d/3
- Real models choose and round different widths

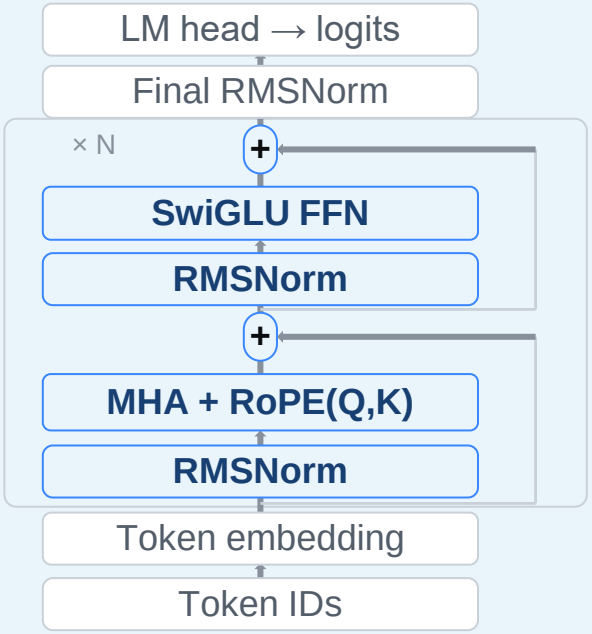
$$2d(4d) = 3d(8d/3) = 8d^2$$

MODEL	FFN	d_ff/d
Original Transformer	ReLU	4.00
Phi-3 Mini	SwiGLU	2.67
Llama 3 8B	SwiGLU	3.50
Mistral 7B	SwiGLU	3.50
Gemma 2 9B	gated GELU	4.00
Qwen2.5 7B	SwiGLU	5.29

Ratio alone is insufficient: check matrix count, rounding, and whether experts are routed.

Publisher configurations — references at end

Summary for Transformer Blocks



one block · input and output [B,T,d]

COMPLETE COMPUTATION

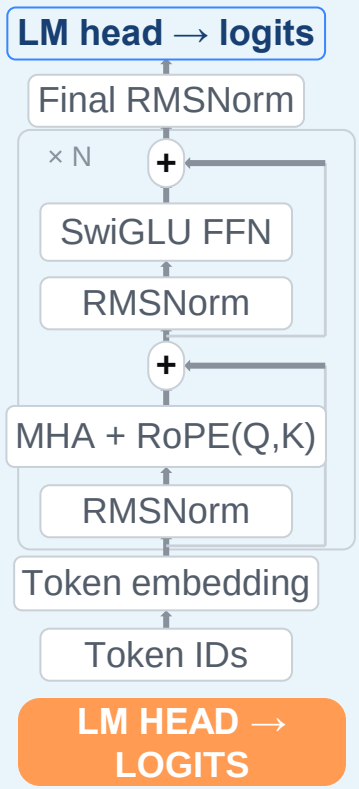
Attention mixes positions; the FFN transforms features.

- RMSNorm → MHA with RoPE → residual addition
- RMSNorm → SwiGLU → residual addition
- Both updates return to [B,T,d]
- The next block receives exactly the same outer shape

$$x' = x + \mathrm{MHA}(\mathrm{RMSNorm}(x))$$

$$y = x' + \mathrm{SwiGLU}(\mathrm{RMSNorm}(x'))$$

Output Logits



OUTPUT PROJECTION

Project the final-position hidden state from d features to V logits.

- Take h_T [B,d] after the final RMSNorm
- Multiply by W_U [d,V] → logits [B,V]
- Logits are scores; they need not sum to one
- With weight tying, $W_U = E^T$

```
z = h_T W_U     h_T:[B,d],
    W_U:[d,V], z:[B,V]
```

final stream
[1,16,128]

take position T
[1,128]

LM head
[128,2048]

logits
[1,2048]

Tied $W_U = E^T$; output multiplication still costs FLOPs.

INPUT	OUTPUT	PARAMETERS	FLOPs
h_T [1,128]	z [1,2048]	0 extra — tied to E^T	2BdV=0.524M

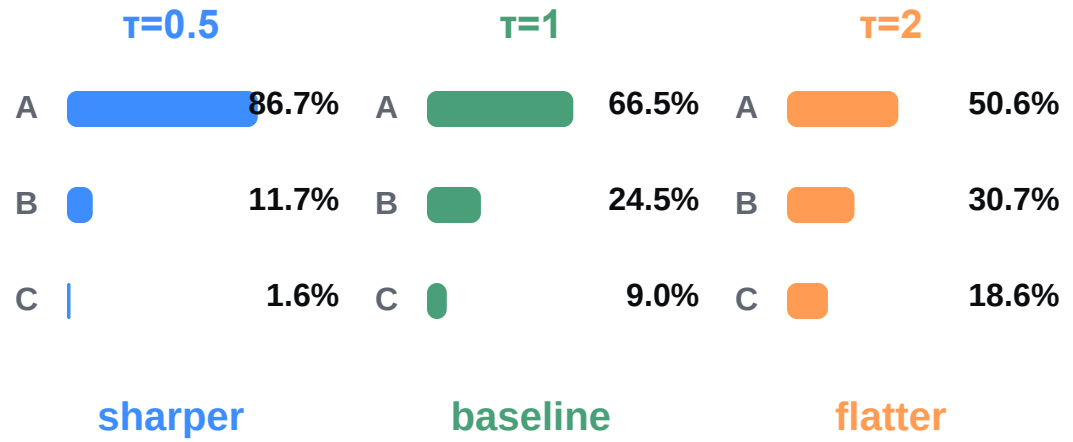
Temperature

DECODING HYPERPARAMETER

Divide logits by positive τ, then apply softmax.

- Low τ — sharper distribution
- High τ — flatter distribution
- Model weights and original logits stay fixed
- Positive τ preserves the greedy argmax

$$P(i)=\exp(z_i/\tau) \,/\, \Sigma_j \exp(z_j/\tau)$$



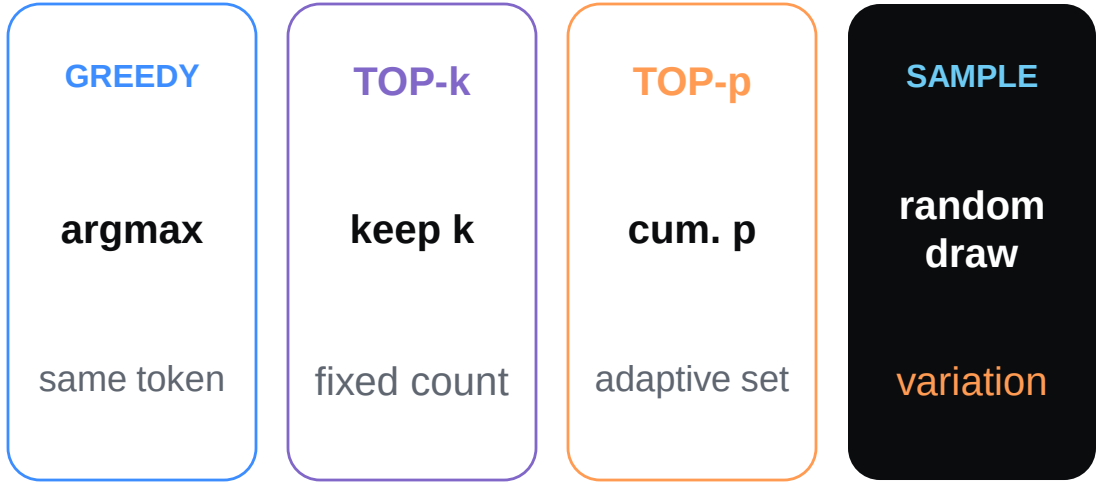
Check — can a positive temperature change the greedy choice?

Sampling

TOKEN SELECTION

Different decoding rules can produce different continuations.

- Greedy — choose the largest logit
- Top-k — retain k highest-probability candidates
- Top-p — retain the smallest set reaching mass p
- Renormalize retained probabilities, then sample



temperature → top-k/top-p → renormalize → sample

Once one sampled token differs, it becomes part of the next prefix and can change the remainder of the generated sequence.

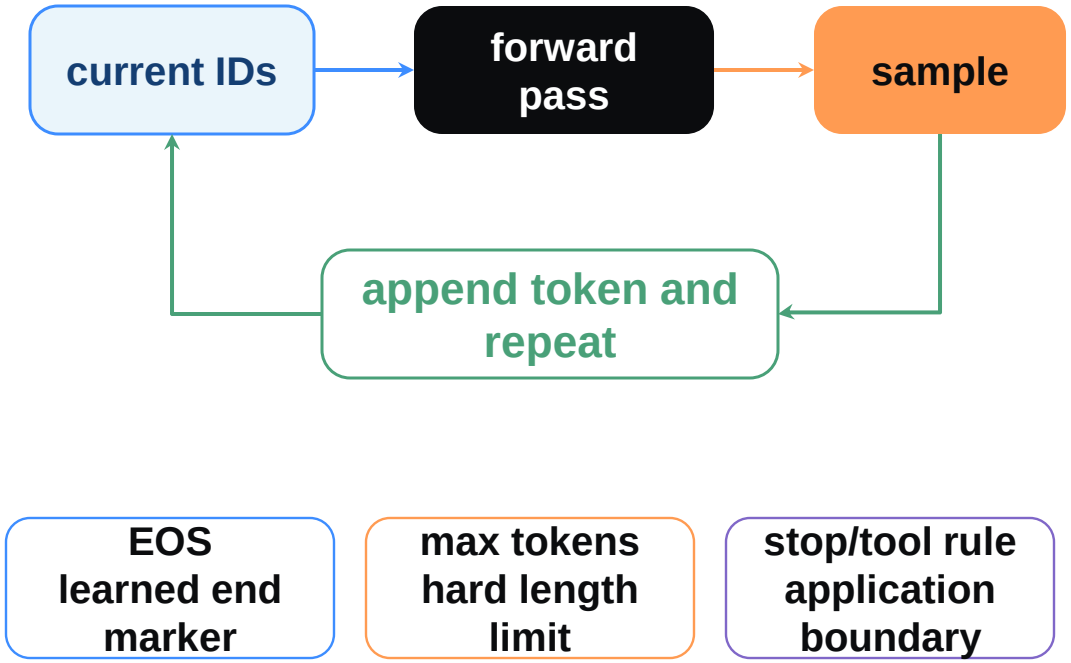
Nucleus sampling: Holtzman et al. (2020)

Auto-regressive Generation

AUTOREGRESSIVE GENERATION

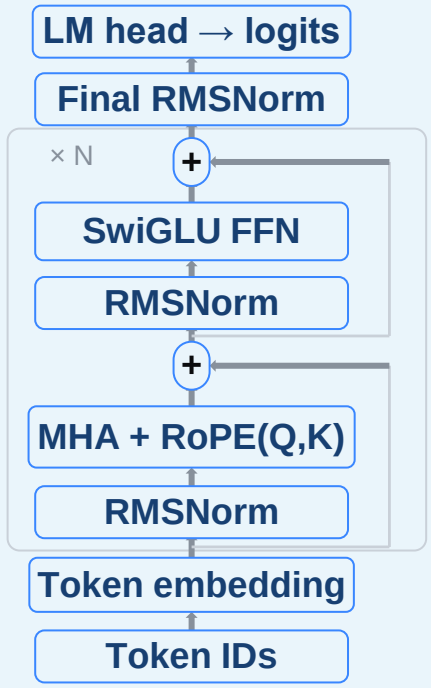
Append the selected ID and run the model again.

- Each pass produces a next-token distribution
- The selected ID becomes part of the next prefix
- EOS or an application rule ends generation
- The same model weights are reused every step



The vanilla loop may repeat earlier computation. KV-cache reuse is the central optimization introduced next lecture.

One complete inference pass



complete decoder-only pass

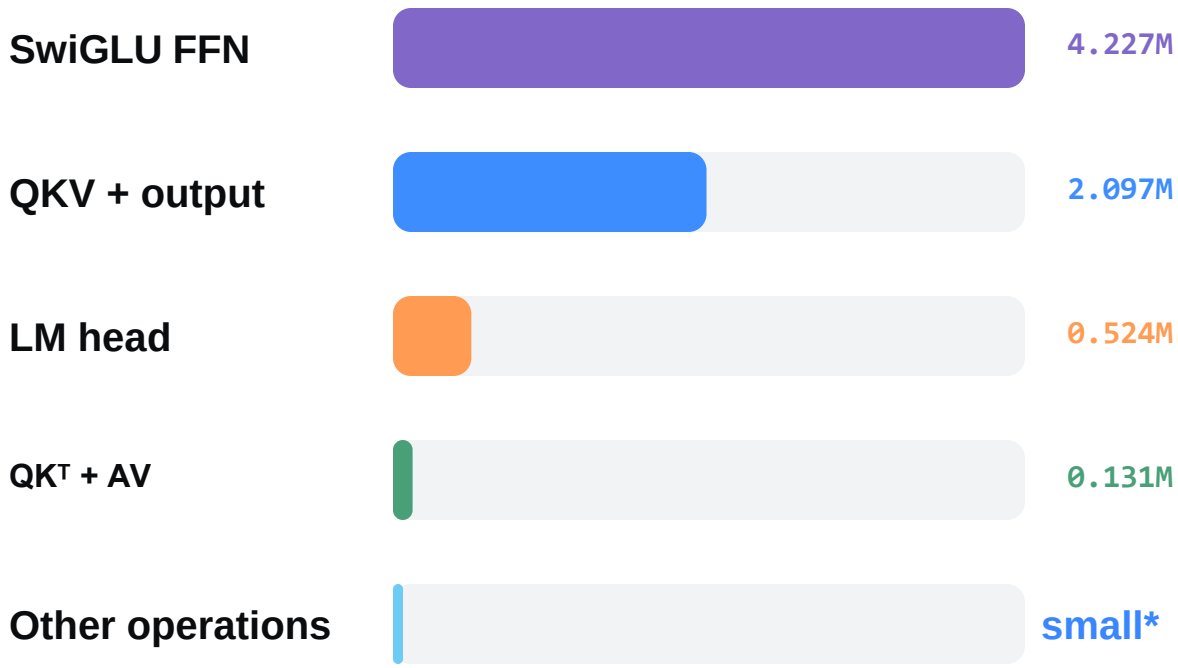
1	**Tokenize**	`text → IDs [B,T]`	BPE applies fixed merges
2	**Represent**	`IDs → X [B,T,d]`	embedding; position enters attention
3	**Attention**	`[B,T,d] → [B,T,d]`	RMSNorm → QKV → RoPE → A V → W_O → add
4	**Activation**	`[B,T,d] → [B,T,d]`	RMSNorm → SwiGLU → residual add
5	**Score**	`h_T → logits [B,V]`	final norm and LM head
6	**Select**	`probabilities → token ID`	temperature, filtering and sampling

Compare computation across the layers

TOY COST MODEL

For this short prompt, FFN and dense projections dominate.

- Main example — B=1, T=16, d=128, N=1
- SwiGLU width 344; final-position LM head only
- Count multiply and add separately
- FLOPs describe arithmetic, not measured latency



toy total ≈ 7.0 MFLOPs

*Norms, RoPE, softmax, activation and additions omitted from the leading matrix total.

Count stored weights as well as FLOPs

SAME TOY MODEL

Large weight storage does not imply large arithmetic cost.

Embedding and LM head share one matrix
Attention — four d × d matrices
FFN — two d × d_ff; one d_ff × d
Three RMSNorm scales; no linear biases

Component	Parameters
Embedding / tied head	262,144
MHA projections	65,536
SwiGLU FFN	132,096
RMSNorm scales	384
Total	**460,160**

FP16 / BF16 weights ≈ 0.92 MB
FP32 weights ≈ 1.84 MB

Untied head adds 262,144 weights. These are decimal MB of weights only; runtime buffers need extra memory.

Architecture hyperparameters

ENGINEERING CHOICES

Choose dimensions together, then validate experimentally.

- V trades vocabulary storage against token length
- d, N, and d_ff allocate capacity and computation
- H and d_h define the attention layout
- Data, hardware, and task determine useful choices

KNOB	AFFECTS	COST	NOTE
V	embedding / LM head	$\propto Vd$	may reduce T
T	token-wise + attention	$\propto T$ and T^2	context cost
d	most dense matrices	$\propto d^2$	width and heads
N	repeated blocks	$\propto N$	depth
d_ff	FFN capacity	$\propto d \cdot d_ff$	2 vs 3 matrices
H, d_h	attention layout	$d = H \cdot d_h$	grouping later

Part science: derive effects. Part engineering: choose and validate under constraints.

V and T trade off; width and depth allocate capacity. Evaluate the complete configuration.

Next — faster inference and smaller memory use

LOOKING AHEAD

Separate exact execution changes from architecture trade-offs.

- KV cache and exact kernels reuse or reorganize computation
- GQA, MQA, and MLA change attention design
- Quantization reduces bytes with numerical approximation
- Measure quality, latency, throughput, and memory

CAUSAL MASK
visibility

KV CACHE
reuse

GQA / MQA
share K,V

MLA
compress state

QUANTIZATION
fewer bytes

KERNELS
better execution

BENCHMARKS
measure reality

References — language modelling and architecture

Shannon (1951)
Prediction and Entropy of Printed English

Bengio et al. (2003)
A Neural Probabilistic Language Model

Mikolov et al. (2013)
Efficient Estimation of Word Representations

Vaswani et al. (2017)
Attention Is All You Need; original architecture figure

Radford et al. (2018)
Improving Language Understanding by Generative Pre-Training

Kaplan et al. (2020)
Scaling Laws for Neural Language Models

Hoffmann et al. (2022)
Training Compute-Optimal Large Language Models

Gage (1994); Sennrich et al. (2016)
BPE compression and neural translation with subwords

Hashimoto — Stanford CS336
Supplied Lecture 3 figures; architecture teaching reference

Click a linked title for the source. Numerical examples in this lecture are illustrative calculations.

References — components and model configurations

Ba et al. (2016); Zhang & Sennrich (2019)
LayerNorm and RMSNorm

Su et al. (2021)
RoFormer — Rotary Position Embedding

Xiong et al. (2020)
On Layer Normalization in the Transformer Architecture

Shazeer (2020); PyTorch SiLU
GLU Variants Improve Transformer; SiLU = Swish at β=1

Holtzman et al. (2020)
The Curious Case of Neural Text Degeneration

Ainslie et al. (2023)
Grouped-query attention and checkpoint uptraining

Llama 3 8B; Mistral 7B
Publisher configurations — width, depth, FFN dimensions

Phi-3 Mini; Gemma 2 9B
Publisher configurations — FFN comparison

Qwen2.5 7B
Publisher configuration — d=3584; d_ff=18944

Click a linked title for the source. Numerical examples in this lecture are illustrative calculations.